

Construction Replay Transcript

CSV SFTP To SQL · 2026-08-12 · what you'll hear in documents/construction-replay.mp4

What this interface does

Poll a CSV file from an FTP directory, stage it locally, parse rows (CSV to XML Dialect A), and insert into SQL Server `dbo.CsvPatients`.

Who it's for

- Ops and trading partners dropping CSV files on FTP
- Stakeholders who want the Capability Brief, not route XML
- Engineers running the Test Plan against the Demo UI and SQL
- Anyone watching the construction video or reading this transcript

Systems involved

- Trading partner / ops drop CSV on FTP (demo / demo, dir upload)
- PilotFish eiPlatform (23R1)
- SQL Server 2022 (CsvSftpDemo)

Documents we also produce

csv_sftp_to_sql_Capability_Brief.pdf · ready

Short overview of what the interface does — shareable without reading route XML.

CSV_SFTP_To_SQL_Test_Plan.pdf · ready

Scenarios that prove the interface works end-to-end (2 cases in tests/plan.json).

Module documentation pack (7 deep-dive PDFs) · ready

PilotFish product PDFs for the modules used in this build.

construction-replay.mp4 + this transcript · ready

Narrated walkthrough of building the routes, then a live inject to SQL smoke test.

Narration

Spoken lines from the construction video (setup, build-replay, live test, closing).

Setup

Welcome

Welcome to the PilotFish demo: CSV FTP To SQL. This one didn't need any custom code — PilotFish supports the whole flow out of the box.

Pipeline overview

Before we wire anything, here's the flow end to end. A CSV lands on FTP, we stage it, turn the rows into XML, and load them into SQL Server. Two routes — pickup, then load.

External systems

And here's what sits around the routes. FTP holds the drop folder, SQL Server holds the patients table, eiPlatform runs the interfaces, and this Web UI is where we inject and review. All of that is Docker for the demo.

Blank canvas

Here is a blank canvas, so let's get started creating the new PilotFish interface.

What OGNL is

One thing you'll see in a few configs: OGNL. Think of it as a tiny expression language — instead of hard-coding a file name, we build it from the transaction. On archive and stage we use the original name, an underscore, a timestamp, then .csv. That keeps every copy unique without losing where it came from.

1 - SFTP Poll And Stage

Pickup first

Ops drops a patient CSV and we land the rows in the database — no hand-loading. Two routes: pickup first, then the load. Starting with pickup: grab the file from FTP, keep a raw archive, and stage a local copy — before we ever touch the database.

FTP listener

First up is the FTP listener — it watches the upload folder on the FTP server. We're only taking .csv files. It checks about every 10 seconds. Once we've got the file, we delete it remotely so it doesn't get picked up again.

Archive step

Next we archive the raw file — an exact copy of whatever arrived, before we touch the data. That lands under /opt/pilotfish/output/archive, original name plus a timestamp.

Stage-to-disk step

Then we stage a local copy for the next route, writing to /opt/pilotfish/input/staged, so FTP pickup stays separate from the database work.

2 - CSV To SQL

The load route

Onto the load route — this one never talks to FTP. It watches the staged folder, turns each CSV into SQL, and loads the patients table.

Staged-folder listener

Next is the staged-folder listener — it never talks to FTP, just the local stage. It watches /opt/pilotfish/input/staged. Same idea: only .csv files. It polls every 5 seconds. After pickup we move the file aside so the folder stays clean.

CSV processor

Here's the CSV processor — it turns the file into XML and uses the header row for the column names.

Mapping step

Now for the mapping — I'll open the stylesheet. We're using the stock XSLT processor with csv-to-sqlxml.xslt. Watch the for-each over each CSV record and the column mappings: Dialect A tags like PATIENTID into the SQL insert fields, and STATE becomes StateCode.

SQL insert

And finally we insert into SQL — those inserts go into the demo database over JDBC. That's it: CSV in, patients in SQL.

Live test

After construction, the video switches to the Demo tab and runs inject → SQL.

Prove it works

Routes are built. Let's switch to the Demo tab and prove it works.

Drop patients.csv on FTP

I'll drop patients.csv into the FTP upload folder — same place a trading partner would.

Waiting for the routes

Give the routes a moment. Route one picks up the file and stages it; route two parses the CSV and inserts into SQL.

Rows in SQL

There they are — patient rows in the database. File in on FTP, data out in SQL.

Closing

Demo complete

That's the demo — file in on FTP, patients in SQL. Thanks for choosing PilotFish.